SchoolNet AI Code and Project Reference

Brief guide to the folders, files, functions, database, workflow, and technologies

Purpose: This document explains how the SchoolNet AI network-threat investigation project is organised and how its major parts work together. It is designed for project explanation and viva preparation. It excludes the .env file because that file contains local configuration and secret values.

1 Project at a glance

Layer	What it does
Web application	
	Flask serves pages, receives PCAP uploads, keeps the selected capture in session state, and starts the analysis pipeline.
Evidence storage	
	SQLite stores analyses, packets, rule alerts, timeline events, flows, anomalies, incidents, devices, policies, reports, and AI messages.
Detection	
	Rule-based detectors identify known suspicious patterns. Isolation Forest compares new flow behaviour with the trained normal baseline.
Investigation	
	Correlation groups evidence by device and time. Reports, protocol findings, device roles, and AI assistance help explain results.
Security	
	PCAP and report integrity use SHA-256. Protected reports use AES-256-GCM and optional RSA-PSS signatures.

2 End to end workflow

- User uploads a PCAP or selects an earlier capture. app.py creates an Analysis record and saves a separate working copy in uploads.
- Packet parser reads the PCAP and stores packet fields. Rule detectors create ThreatEvent records.
- SchoolNet enrichment builds flows, evaluates policy context, scores flows against the active ML baseline, and correlates evidence into incidents.
- Dashboard pages show the selected Analysis ID. The same ID connects packets, flows, anomalies, incidents, reports, and AI context.
- A protected report can be generated from current evidence. Its trusted SHA-256 digest and RSA-PSS metadata are stored at creation time.

3 Top level folders and files

Path	Role
app.py	
	Main Flask app. Upload route, background analysis pipeline, original dashboard routes, deletion safety, and application startup.
config.py	
	Central non-secret settings: folders, upload size, flow window, thresholds, AI provider choice, and database location.
manage.py and run.py	
	Convenience entry points for database setup and application execution.

requirements.txt	
	Python packages required by the project.
analysis	
	Original PCAP parsing, rule detection, scoring, correlation, timeline, and traffic-statistics code.
schoolnet	
	SchoolNet AI extension: flows, ML, incidents, protocols, secure reports, health check, authentication, routes, and AI integration.
database	
	SQLAlchemy setup, original data models, SchoolNet data models, and additive migrations.
templates	
	Jinja HTML pages for the original investigator and SchoolNet screens.
static	
	CSS visual theme plus browser JavaScript for upload progress, charts, forms, chat, and verification requests.
reports	
	ReportLab PDF generator.
ai	
	Prompt templates and deterministic/AI investigation helpers.
utils	
	Input validation and general helpers.
sample_data	
	Small built-in generated demos and metadata used for development/testing.
tests	
	Automated regression tests.
uploads	
	Private working copies of analysed PCAP files. These are not the user test-library folder.
models_store	
	Saved Isolation Forest joblib model artifacts.
reports_output	
	Protected report archives: PDF or encrypted SNA files.
keys	
	Local signing-key location. Private key material must not be shared.
output	
	Generated user documentation and screenshots.
tmp	
	Temporary generation/render-check files; not part of application logic.
pcap_investigator.db	
	Local SQLite database holding the analysis records.
vercel.json	
	Deployment configuration.

4 Analysis and detection code

File	Important functions	Brief responsibility
analysis/pcap_parser.py		
	parse_pcap; packet extraction helpers	
		Reads PCAP packets through PyShark with a Scapy fallback; extracts IP, ports, DNS,

		HTTP, TLS, flags, and timing.
analysis/threat_detector.py		
	detect_port_scans; detect_ssh_brute_force; DNS and web/TLS detectors	
		Runs independent rule checks and returns structured event dictionaries. Thresholds come from config.py.
analysis/threat_scorer.py		
	score_threats	
		Applies risk/severity scoring to detector output.
analysis/correlator.py		
	correlate_events	
		Groups related rule detections into attack activity.
analysis/timeline_builder.py		
	build_timeline	
		Creates the chronological evidence timeline.
analysis/traffic_analyzer.py		
	get_traffic_stats	
		Produces traffic totals and protocol statistics for screens and reports.
analysis/__init__.py		
	Package marker	
		Makes analysis importable as a Python package.

5 SchoolNet AI code

File	Important functions	Brief responsibility
schoolnet/routes.py		
	selected; reports; policies; incidents; devices	
		SchoolNet page/API routes. selected keeps the capture selection stable through session and analysis_id.
schoolnet/pipeline.py		
	initialize_context; enrich_analysis; file_hash	
		Coordinates flows, model scoring, rules, and incident correlation after parsing.
schoolnet/flows.py		
	build_flows; policy_finding_for_flow; refresh_policy_findings	
		Builds bidirectional time-window flows, snapshots device roles, and evaluates approved communication policies without altering packets.
schoolnet/ml.py		
	train_baseline; score_analysis	
		Trains and persists an Isolation Forest model; scores only separate captures to prevent baseline data leakage.
schoolnet/incidents.py		

	correlate_incidents	
		Combines rule alerts, ML anomalies, and policy findings. ML evidence remains distinct from rule and policy evidence.
schoolnet/protocols.py		
	protocol_summary	
		Summarises observed protocols and available TLS metadata.
schoolnet/crypto.py		
	encrypt; decrypt; sign; verify_signature; create_report; verify_uploaded_report	
		Creates protected reports and compares uploaded report copies to trusted stored metadata; never replaces the archive.
schoolnet/ai_engine.py		
	ask_provider; investigate; chat	
		Builds evidence context and calls the configured AI provider when available; retains an offline fallback.
schoolnet/auth.py		
	Authentication helpers	
		Provides SchoolNet login/session protection.
schoolnet/health.py		
	System checks	
		Checks database, model, report-security, and AI readiness for /check.
schoolnet/__init__.py		
	Package marker	
		Makes SchoolNet modules importable.

6 Database design

The project uses Flask SQLAlchemy with a local SQLite database. database/database.py creates and connects tables. database/migrations.py applies small additive compatibility migrations when the app starts.

Model group	Tables and purpose
Original evidence models	
	Analysis is one capture. Packet stores parsed network evidence. ThreatEvent stores rule detections. TimelineEvent stores ordered events. Investigation stores the report-style investigation summary.
SchoolNet context	
	AnalysisContext stores capture SHA-256, stage/progress, baseline state, and provenance. Flow stores aggregated connections. DeviceWindow stores behaviour in a time window.
ML and incident models	
	ModelRecord stores active model metadata. Anomaly stores per-flow model results. Incident stores correlated incident evidence, reason, risk, analyst status, and confirmation.
Operations models	
	Device stores registered IP/name/role. CommunicationPolicy stores an approved role-to-destination relationship. ChatMessage stores assistant conversation.
Report model	
	SecureReport stores the original report filename, trusted SHA-256, encryption status, detached signature, and matching public key.

7 Secure report workflow

- create_report generates a PDF from the selected analysis. It hashes the original PDF, optionally signs it with RSA-PSS, optionally encrypts the stored file with AES-256-GCM, and saves trusted metadata in SecureReport.
- Encrypted reports are stored as .sna. Their raw encrypted copy must be downloaded and uploaded again for exact-copy verification. A PCAP file is not a report and cannot pass report verification.
- verify_uploaded_report decrypts an encrypted copy only when authentication succeeds, then compares the PDF SHA-256 with the saved trusted hash and verifies the saved RSA-PSS signature using the saved public key.
- Verification input is read in memory only. It never overwrites, deletes, or changes the archived report, selected capture, packets, or analysis results.

8 Device policies and incidents

When an approved relationship is saved, the selected capture is reevaluated through refresh_policy_findings and correlate_incidents. This updates only policy findings. Rule-based evidence and ML anomaly evidence are deliberately not removed because they are independent reasons to investigate.

Signal type	Meaning	Effect of approved policy
Rule		
	Known suspicious pattern such as port scan or SSH brute-force behaviour.	
		No automatic removal.
ML anomaly		
	Flow differs from normal baseline behaviour.	
		No automatic removal.
Role-policy finding		
	Repeated Student PC access to a registered sensitive device without an approved relationship.	
		Removed from active findings when the relationship matches the saved policy; historical evidence remains labelled.

9 Frontend and report files

Folder/file	What it contains
templates/base.html	
	Shared navigation, capture strip, session/CSRF metadata, and page layout.
templates/school_overview.html through school_settings.html	
	SchoolNet dashboard, flows, devices, anomaly, incident, AI, protocol, reports, health, login, and settings screens.
templates/dashboard.html, events.html, packets.html, timeline.html, investigation.html, report.html, upload.html, analyses.html, about.html, error.html, index.html	
	Original investigator pages for browsing evidence, reports, uploads, and errors.
static/css/style.css	
	Shared dark SOC visual style and responsive layout.
static/js/schoolnet.js	
	Upload progress, processing polling, device form population, AI chat, report archive checks, and uploaded-copy verification.

static/js/dashboard.js and timeline.js	
	Charts and timeline behaviour for the original investigator pages.
reports/pdf_report.py	
	ReportLab layout, severity colour mapping, PDF sections, and investigation-report rendering.

10 Libraries used

Library	Use in this project
Flask, Werkzeug	
	Web server, routes, sessions, request handling, file uploads, and secure filename handling.
Flask SQLAlchemy and SQLAlchemy	
	SQLite ORM, models, queries, and relationships.
PyShark and Scapy	
	PCAP decoding and fallback packet parsing.
scikit-learn, NumPy, joblib	
	Isolation Forest ML, feature preprocessing, arrays, and saved model files.
cryptography	
	AES-256-GCM authenticated encryption, RSA keys, RSA-PSS signatures, and signature validation.
ReportLab	
	PDF report creation.
google-genai and OpenAI	
	Optional external AI investigation/chat provider integrations.
requests	
	HTTP support for provider/API calls.
pytest	
	Automated tests for parser, detection, ML, security, upload, capture-selection, and policy behaviour.
python-dotenv	
	Loads local environment settings without placing secrets in source code.

11 Test and data folders

Location	Use
TEST_PCAP_FILES outside the app folder	
	Main presentation/test library: normal baseline, normal evaluation, cyber attack incident, and mixed-threat PCAPs.
sample_data/schoolnet	
	Built-in smaller test PCAPs and JSON provenance/evaluation metadata.
tests/test_detection.py	
	Original parser/detector/report regression tests.
tests/test_schoolnet.py	
	Flow, policy, ML, protocol, crypto, AI fallback, and incident-correlation tests.
tests/test_upload_selection.py	
	Full upload-to-results scenario: selection state, reports, verification copies, policies, and deletion safety.
tests/test_ai_connection.py	

	AI provider connection/error behaviour tests.

12 What to say in a presentation

- A PCAP is uploaded once and becomes an Analysis. The Analysis ID keeps every packet, detection, flow, anomaly, incident, and report connected.
- The system uses two detection styles: deterministic rules for known attack patterns and Isolation Forest for unusual traffic relative to a normal baseline.
- The normal baseline is trained once and saved. Test/cyber/mixed captures are evaluated against it; training captures are excluded from scoring to prevent data leakage.
- Device policies only approve legitimate communication relationships. They do not erase separate ML or rule evidence.
- Secure reports protect confidentiality with AES-GCM, integrity with SHA-256, and authenticity with RSA-PSS. Verification uses trusted metadata created with the original report.

Security note: The .env file is intentionally not documented here. It is the local location for keys and deployment-specific settings and should not be shared.